Assignment - 5

Topics Covered: File Handling in Python (Opening, Reading, Writing, Appending, Modes, Paths), Error Handling & Logging, and NumPy Foundations

Question 1: Explain the different file modes available in Python's open() function ("r", "w", "a", "x", "r+", and
others), including what happens for each mode if the target file does not already exist. Then explain the difference between .read(), .readline(), .readlines(), and looping directly over a file object, including which
approach is most memory-efficient for very large files and why. Support your answer with a short example that
opens a text file and reads its content using two different methods.

Answer:

1. File modes in python's open() Function

   Python's open() function is used to open a file for reading, writing, or appending data. The file mode specifies how the file will be accessed.

   Different file modes

Mode	Description	If the file does not exist
"r"	Opens a file for reading.	Raises FileNotFoundError.
"w"	Opens a file for writing. Existing content is erased	Creates a new file.
"a"	Opens a file for appending data at the end.	Creates a new file.
"x"	Creates a new file for exclusive writing. Fails If the file already exists.	Creates a new file.
"r+"	Opens a file for both reading and writing without truncating existing content.	Raises FileNotFoundError.
"w+"	Opens a file for both reading and writing. Existing content is erased.	Creates a new file.
"a+"	Opens a file for reading and appending.	Creates a new file.

“rb”	Opens a file for reading in binary mode.	Raises Creates a new file.
“wb”	Opens a file for writing in binary mode. Existing content is erased.	Creates a new file.
“ab”	Opens a file for appending in binary mode.	Creates a new file.

2. **Difference Between File-Reading Methods**

a) read()

- Reads the entire file content as a single string by default.
- You can specify the number of characters to read, such as read(10).
- It may consume a large amount of memory when used on very large files.

b) readline()

- Reads one line at a time.
- Each call returns the next line, including its newline character when present.
- It is useful when you need to process a file line by line.

c) readlines()

- Reads all lines of a file and returns them as a list of strings.
- Each list element represents one line.
- It can consume considerable memory for large files because all lines are stored in a list.

d) Looping directly over a file object

- Uses a for loop to read and process one line at a time.
- Does not require loading the entire file into memory.
- It is generally the most memory-efficient approach for processing very large text files because
  Python reads lines incrementally.

3. **Example :** Reading a Text file using Two Methods

Suppose a text file named sample.txt contains:

Hello, Python!
Welcome to file handling.
This is an example.

Method 1: Using read()

```
With open("sample.txt", "r") as file:
        content = file.read()
        print(content)
```

Output:

```
Hello, Python!
Welcome to file handling.
This is an example.
```

Here, read() reads the entire file content into a single string.

Method 2: Using a for loop

```
with open("sample.txt", "r") as file:
        For line in file:
                print(line, end="")
```

Output:

```
Hello, Python!
Welcome to file handling.
This is an example.
```

Question 2: Explain the difference between opening a file in "w" mode and "a" mode, and why "w" mode can accidentally destroy existing data if used carelessly. Then explain why every file you open should eventually be closed, what can go wrong if you rely only on a manual f.close() call, and how the with statement solves this problem — even when an exception is raised inside the block. Write a short program that writes a list of at least three strings to a file using write() or writelines(), and then appends one more line to the same file without erasing the earlier content.

Answer:

Difference between w mode and a mode

- w (write) mode: Opens a file for writing. If the file already exist, its previous content is erased (truncated). If the file does not exist, a new file is created.
- a (append) mode: Opens a file to add new content at the end. It preserves the exist content and creates the file if it does not exist.

Why can w mode destroy data?

If we can a file containing important information using w mode, python immediately erases the existing content. If we use it carelessly, the original data may be lost.

Why should every file be closed?

Closing a file releases system resources, ensures buffered data is written to the file, and prevents unnecessary use of file handles.

If we rely only on a manual f.close() call, an exception or error occurring before that statement may prevent the file from closing prperly.

The with statement solves this problem by automatically closing the file when the block ends, even if an exception is raised inside the block. It makes file handling safer and cleaner.

Python Program

```
lines = [“Apple \n”, “Banana \n”, “Mango \n”]

with open(“fruits.txt”, “w”) as f:
        f.writelines(lines)

With open(“fruits.txt”, “a”) as f:
        f.write(“Orange \n”)

With open(“fruits.txt”, “r”) as f:
        print(f.read())
```

Output:

```
Apple
Banana
Mango
Orange
```

Question 3: Explain the difference between a relative path and an absolute path when opening a file. In the
specific context of Google Colab, explain where a file created with just a filename, such as open("sample.txt",
"w"), is actually saved, why that file disappears when the runtime resets, and how mounting Google Drive
changes this behaviour. Write a short program that creates a file at a normal (non-Drive) path, writes some
content to it, and then reads and prints that content back using a with block.

Answer:

Difference Between Relative Path and absolute Path

- Realtive Path: A path that specifies the location of a file relative to the current working dictionary. For example, open(“sample.txt”, “w”) creates a file named sample.txt in the current dictionary.

- Absolute Path: A complete path that specifies the exact location of a file. For example, /content/sample.txt specifies a file in the /content dictionary in Google Colab.

File Storage in Google Colab

In Google colab, a file created open(“sample.txt”, “w”) is normally saved in the current working directory, which is usually /content.

This file is stored in the temporary virtual machine environment provided by Colab. When the runtime resets or is deleted, the file may be lost because the temporary storage is not permanent.

Mounting Google Drive allows us to access files in Drive, which provides persistent storage. Files saved in a mounted Drive folder, such as
/content / drive / MyDrive/, remain available even after the Colab runtime resets.

Python Program

The following program creates a file at a normal, non-Drive path, writes content to it, and them reads and print the content using with blocks.

```
# Creates and write to a file

With open(“ /content / sample.txt”, “w”) as f:
        f.write(“Hello, welcome to Python programming!”)

# Read and print the file content
```

```
With open(“/content / sample.txt”, “r”) as f:
        content = f.read()
        print(content)
```

Output:

Hello, welcome to Python programming!

Question 4: Look at the code below and answer the questions:

```
def divide(a, b):
        Try:
                result = a / b
        except  ZeroDivisionError:
                print("Cannot divide by zero!")
        else:
                print("Division successful:", result)
        finally:
                print("Divide operation attempted.")
divide(10, 2)
divide(10, 0)
```

Explain the difference between syntax errors, runtime errors (exceptions), and logical errors in Python, giving one
short example of each. Then explain exactly what will be printed for each call to divide() above, and describe
precisely when the else block runs compared to when the finally block runs.

Answer:

1. Difference Between Syntax Errors, Runtime Errors, and Logical Errors

    a) **Syntax Errors:** These occur when Python rules are violated, so the program cannot run properly.

    Example:

    Print(“Hello”

    Here, the closing parenthesis is missing.

    b) **Runtime Errors (Exceptions):** These occurs while the program is executing. They interrupt normal exception unless handled.

Example:

print(10 / 0)

This raises a ZeroDivisionError because division by zero is not allowed.

c) **Logical Errors:** These occur when a program runs without crashing but produces an incorrect result due to faulty logic.

Example:

```
length = 5
width = 3
area  = length + width
print(area)
```

Output: 8

The correct formula for the area of a rectangle is length * width, so the correct answer should be 15.

2. Output of the Given Program

Correctly formatted code:

```
Def divide (a, b):
    try:
        result = a / b
    except  ZeroDivisionError:
        print(“Cannot divide by zero!”)
    else:
        print(“Division successful :”, result)
    finally:
        print(“Divide operation attempted.”)

divide(10, 2)
divide(10, 0)
```

Output:

```
Division successful: 5.0
Divide operation attempted.
Cannot divide by zero!
Divide operation attempted.
```

3. Explanation of Each Function Cell

First call : divide(10, 2)

- The try block executes 10 / 2, producing 5.0.
- No exception occurs, so the except block is skipped.
- The else block runs and prints Division successful: 5.0.
- The finally block runs and print Divide operation attempted.

Second call: divide (10, 0)

- The try block attempts 10 / 0, which raises a ZeroDivisionError.
- The except block handles the exception and prints Cannot divide by zero!.
- The else block is skipped because an exception occurred in the try block.
- The finally block runs and print Divide operation attempted.

4. Difference Between else and finally

- **Else block:** Executes only when the try block completes successfully without raising an exception.

- **finally block:** Executes whether the try block succeeds or an exception is raised and handled. It is commonly used for cleanup operations, such as closing files or releasing resources.

Question 5: Explain why using a bare except: is considered bad practice in real projects, and why you should
always try to catch the most specific exception possible. Then create a custom exception class called InsufficientFundsError that inherits from Exception, and write a function withdraw(balance, amount) that raises
this custom exception (with a meaningful message) whenever amount is greater than balance. Demonstrate
calling this function inside a try/except block that specifically catches InsufficientFundsError, and show what gets
printed both when a withdrawal succeeds and when it fails.

Answer:

1. Why is using a bare except: considered bad practice?

   A bare except: catches almost all exceptions, including exceptions that we may not intend to handle, such as KeyboardInterrupt and SystemExit. This can hide programming errors and make debugging difficult.

We should catch the most specific exception possible because it helps us identify and handle errors correctly without hiding unrelated problems.

For example:

```
try:
        result = 10 / 0
except ZeroDivisionError:
        print(“Cannot divide by zero.”)
```

Here, ZeroDivisionError is caught specifically, making the code clearer and easier to debug.

2. Program Using a Custom Exception

```
class InsufficientFundsError (Exception):
        Pass

def withdraw(balance, amount):
        if amount > balance:
                raise InsufficientFundsError(
                        “Insufficient funds! Withdrawal amount exceeds balance.” )

        balance -= amount
        print(“withdrawl successful.”)
        print(“Remaining balance:”, balance)


# successful withdrawal

try:
        withdraw(1000, 300)
except InsufficientFundsError as e:
        print(e)

# Failed withdrawal

try:
        withdraw(1000, 1500)
except InsufficientFundsError as e:
        print(e)
```

3. Output

Withdrawal successful.
Remaining balance: 700
Insufficient funds! Withdrawal amount exceeds balance.

4. Explanation

    - Customer exception class: InsufficientFundsErrors inherits from Python's built-in Exception class, allowing us to define a specific error for insufficient funds.

    - raise statement: When the withdrawal amount exceeds the available balance, Raise generated an InsufficientFundsErrors with a meaningful error message.

    - Successful withdrawal : When the balance is 1000 and the withdrawal amount is 300, the Withdrawal succeeds, leaving a balance of 700.

    - Failed withdrawal : when the withdrawal amount is 1500 but the balance is 1000, the Function raises InsuffucientFundsError. The matching except block catches it and prints the error message.

Question 6: Explain at least three advantages of using Python's logging module over plain print() statements for
tracking what a program is doing. List the five standard log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL) in
order of increasing severity, and give one realistic example of when you would use each. Then write a short
program that uses logging.basicConfig() to write log messages of at least three different levels to a file called
app.log, with each log line including a timestamp and the severity level.

Answer:

1. Advantages of Using Python's logging Module over print()

Python's logging module is more useful than plain print() statements for tracking program execution for the following reasons:

a) Different severity levels: Logging provides level such as DEBUG, INFO, WARNING, ERROR, and CRITICAL to indicate the importance of a message.
b) Writes to files: Log message can be saved to a file for future reference, whereas print() normally display message on the console.
c) Timestamps and additional information : Logging can automatically include timestamps, severity levels, and other details in each log message.

d) Configurable output: We can control which message are displayed or recorded by setting a minimum logging level.
e) Easier debugging and maintenance : Logs help developers identify errors and understand what happened during program execution.

2. Five Standard Logging Levels

The following levels are listed in order of increasing severity:

Log Level	Severity	Example of Use
DEBUG	10	Recording variable values while debugging a calculation.
INFO	20	Recording that a user has successfully logged in.
WARNING	30	Warning that disk space in running low
ERROR	40	Recording a failed database connection.
CRITICAL	50	Recording a situation where the application cannot continue running.

3. Python Program to write Logs to app.log

```
# Configure logging to write to a file

logging.basicConfig(
        filename=”app.log”,
        level=logging.DEBUG,
        format=”%(asctime)s - %(levelname)s - %(message)s”
)

# Log message at different levels

logging.debug(“Debugging application settings.”)
logging.info(“Application started successfully.”)
logging.warning(“Disk space is running low.”)
logging.error(“Failed to cannot to the database.”)
logging.critical(“Application cannot continue.”)
```

4. Example Contents of app.log

The file will contain lines similar to the following (timestamps will vary):

2026-10-14 08:30:00,123 - DEBUG - Debugging application settings.
2026-10-14 08:30:00,124 - INFO - Application started successfully.
2026-10-14 08:30:00,125 - WARNING - Disk space is running low.
2026-10-14 08:30:00,126 - ERROR - Failed to connect to the
database.
2026-10-14 08:30:00, 127 - CRITICAL - Application cannot continue.

5. Explanation of the Program

- Import logging import Python's built-in logging module.
- filename="app.log" specifies the file where log message will be stored.
- level=logging.DEBUG allows all five standard levels to be recorded because DEBUG is the lowest severity.
- format="%(asctime)s - %(levelname)s - %(message)s" specifies that each line includes the timestamp, severity level, and message.
- The five logging methods record message according to their respective severity levels.

Question 7: Look at the code below and answer the questions:

```
import logging

logging.basicConfig(filename="errors.log", level=logging.DEBUG)

def load_config(path):
        Try:
                with open(path) as f:
                        return f.read()
        except FileNotFoundError:
                logging.exception("Failed to load config file")
                return None

load_config("does_not_exist.cfg")
```

Explain what gets written to errors.log when load_config() is called on a path that doesn't exist. Then explain the
difference between logging.error() and logging.exception() — specifically, what extra information logging.exception() records that logging.error() does not, and why it should only ever be called from inside an
except block.

Answer:

1. What gets written to errors.log?

When load_config(“does_not_exist.cfg”) is called, Python tries to open the file does_not_exist,cfg. Since the file does not exist, Python raises a FileNotFoundError exception.

The except FileNotFoundError block catches the exception and executes:

logging.exception(“Failed to load config file”)

The writes an error message and the full traceback to errors.log. The traceback contains details about where the exception occurred and the type of exception raised.

Example contents of errors,log:

ERROR : ROOT : Failed to load config file
Tracback (most recent call last):

FileNotFoundError: [Errno 2] No such file or directory: ‘does_not_exist.cfg’

The Exact traceback and file path details may vary dep[ending on the Python environment.

2. Difference Between logging.error() and logging.exception()

- logging.error() records a message at the ERROR severity level. By default, it does not include exception tracback information.

  Example:

  logging.error(“Failed to load config file”)

- logging.exception() also records a message at the ERROR severity level, but it additionally includes the exception tracback when called inside an exception handler.

  Example:

```
try:
        With open(“does_not_exist.cfg”) as f:
                data = f.read()
except FileNotFoundError:
        logging.exception(“Failed to load config file”)
```

3. Why should logging.exception()
   Be Used Only Inside an except Block?

logging .exception() automatically includes exception information the currently handled exception. Therefore, it should be called inside an except block, where the relevant exception is begging handled.

Outside an active exception handler, there is no currently handled exception to report, so the traceback may be missing or unhelpful.

Question 8: Describe at least four different ways to create a NumPy array (for example: from a Python list, and
using np.zeros(), np.ones(), np.full(), np.eye(), np.arange(), or np.linspace()), including one realistic data-science
or machine-learning situation where each would be useful. Then explain what each of the following array
attributes tells you: shape, ndim, size, dtype, and itemsize. Support your answer with a short example that
creates a 2D array and prints all five of these attributes for it.

Answer:

1. Different Ways to Create NumPy Arrays

NumPy is a Python library used for numerical computations, data analysis, and machine learning. It provides several function for creating arrays efficiently.

First, import NumPy:

```
import numpy as np
```

a) Creating an array a Python list

```
arr = np.array([10, 20, 30, 40])
print(arr)
```

Output:

```
[10, 20, 30, 40]
```

Use: storing a dataset of student marks or converting collected data into a NumPy array for analysis.

b) Using np.zeros()

```
arr = np.zeros((2, 3))
```

```
print(arr)
```

Output:

```
[ [0. 0. 0.]
  [0. 0. 0.] ]
```

Use: Initializing a matrix of zero values before storing results in a machine learning algorithm.

c) Using np.ones()

```
arr = np.ones((2, 3))
print(arr)
```

Output:

```
[ [1.  1.  1.]
  [1.  1.  1.] ]
```

Use: Creating an initial array of once for mathematical operations or initializing certain model parameters.

d) Using np.full()

```
arr = np.full((2, 3), 7)
print(arr)
```

Output:

```
[ [7  7  7]
  [7  7  7] ]
```

Use: Filling a matrix with a default value, such as an initial placeholder or a fixed threshold in data processing.

e) Using np.eye()

```
arr = np.eye(3)
print(arr)
```

Output:

```
[ [1. 0. 0. ]
  [0. 1. 0. ]
```

[0. 0. 1.]]

Use: Creating an identity matrix for linear algebra operations, such as regularization in machine learning.

f) Using np.arangge()

```
arr = np.arange(0, 10, 2)
print(arr)
```

Output:

[0 2 4 6 8]

Use: Generating evenly spaced integer values, such as batch indices or sequence numbers for processing data.

g) Using np.linspace()

```
arr = np.linspace(0, 1, 5)
print(arr)
```

Output:

[0. 0.25 0.5 0.75 1.]

Use: Generating evenly spaced values for plotting graphs, testing model parameters, or evaluating a function over a range.

2. NumPy Array Attributes

NumPy array provide attributes that describe their dimensions, data type, and memory requirements.

Attribute	Meaning
shape	Returns the size of the array along each dimension, as a tuple.
ndim	Returns the number of dimension of the array.
size	Return the total number of elements in the array.
dtype	Specifies the data type of the array elements, such as int32 or float64.
itemsize	Return the memory occupied by each elements in bytes.

3. Program to Print All Five Attributes

```
Import numpy as np

# Create a 2D array

arr = np.array( [ [10, 20, 30],
                  [40, 50, 60] ] )

print("Array:")
print(arr)

print("Shape:", arr.shape)
print("Dimensions:", arr.ndim)
print("Total elements:", arr.size)
print("Data type:", arr.dtype)
print("Item size:", arr.itemsize, "bytes")
```

Output:

```
Array:
[ [10, 20, 30]
  [40, 50, 60] ]
Shape: (2, 3)
Dimensions: 2
Total elements: 6
Data type: int64
Item size: 8 bytes
```

4. Explanation of the Output

- Shape (2, 3) : The array contains 2 rows and 3 columns.
- Dimensions 2: The array is two-dimensional.
- Total elements 6: Each element is stored as a 64-bit integer in this example.
- Data type int64 : Each element is stored as a 64-bit integer in this example.
- Item size 8 bytes : Each element occupies 8 bytes of memory.

Question 9: Explain the difference between basic slicing, boolean indexing, and fancy (integer-array) indexing in

NumPy — specifically, which of these return a view of the original array and which return a copy. Using a short
code example, demonstrate a case where modifying a sliced array also changes the original array, and explain
how .copy() can be used to avoid this. Why does this view-versus-copy distinction matter when working with
large datasets?

Answer:

In NumPy, basic slicing, Boolean indexing, and fancy indexing are used to select elements from an array. The main difference is that basic slicing generally returns a view, whereas Boolean indexing and fancy indexing return copies.

1. Basic Slicing: It selects a portion of an array using a range of indices. It generally returns a view of the original array, so modifying the slicied array also modifies the original array.

2. Boolean Indexing: It selects elements based on a Boolean condition. It returns a copy of the selected elements, so modifications do not affect the original array.

3. Fancy(Integer-Array) Indexing: It selects elements using an array of integer indices. It returns a copy, so modifications do not affect the original array.

Code Example

```
import numpy as np

arr = np.array( [10, 20, 30, 40, 50] )

# Basic slicing

sliced = arr[1:4]
sliced[0] = 99
print(arr)

# Output:        [ 10  99  30  40  50]

# Using copy() to avoid changing the original

copied = arr[1:4].copy()
copied[0] = 77
print(arr)
```

#Output: [10 99 30 40 50]

print(copied)

Output: [77 30 40]

Explanation: In the first example, sliced = arr[1:4] created a view of the original array. Therefore, changing slicied[0] to 99 also changes arr[1] . In the second example, .copy() creates an independent array, so changing copied[0] does not affect the original array.

Importance for Large Datasets

The distinction between views and copies is important for the following reasons:

- Memory efficiency: Views do not require a separate copy of the data, saving memory.
- Performance: View are generally faster to create than copies, especially for large arrays.
- Data safety: Using .copy() prevents accidental changes to the original dataset.

Question 10: Explain NumPy's broadcasting rules in your own words, and give a short example of adding a 1D
array to a 2D array where broadcasting is used. Then explain the difference between np.sum(arr, axis=0) and
np.sum(arr, axis=1) for a 2D array, with a short example of each. Finally, explain the difference between reshape(), flatten(), np.hstack(), and np.vstack(), and demonstrate creating a 1D array of 6 elements, reshaping it
into a 2×3 array, and stacking two such arrays vertically using np.vstack().

Answer:

1. NumPy Broadcastng Rules

Broadcasting is a feature in NumPy that allows arithmetic operations between arrays of different shapes without explicitly changing their shapes. NumPy compares the shapes from right to left. Two dimensions are compatible if they are equal or if one of them is 1.

Example:

```
import numpy as np

arr = np.arr( [ [1, 2, 3],
                [4, 5, 6] ] )
```

```
b = np.array( [10, 20, 30] )

print(arr + b)
```

```
Output:          [ [11  22  33]
                   [14  25  36]
```

Here, the 1D array b is broadcast across both rows of the 2D array, allowing element-wise addition.

2. Difference Between axis=0 and axis=1

In a 2D array, axis=0 calculates the sum column-wise, while axis=1 calculates the sum row-wise.

Example:

```
arr = np.array( [ [1, 2, 3],
                  [4, 5, 6] ] )

print(np.sum(arr, axis=0))
print(np.sum(arr, axis=1))
```

```
Output:          [5  7  9]
                 [  6  15]
```

- axis=0: Adds the elements of each column: 1 + 4 = 5, 2 + 5 = 7, and 3 + 6 = 9.

- axis=1: Adds the elements of each row: 1 + 2 + 3 = 6 and 4 + 5 + 6 = 15.

3. Difference Between reshape(),
   flatten(), np.hstack(), and np.vstack()

- reshape(): Changes the shape of an array without changing its elements. It returns a view when possible; otherwise, it may return a copy.
- flatten() : Converts a multidimensional array into a 1D array and always returns a copy.
- np.hstack() : Combines arrays horizontally, joining them along the columns for 2D arrays.
- np.vstack() : Combines array vertically, stacking them row-wise.

Example:

```
import numpy as np

# Create a 1D array of 6 elements
```

```
arr = np.array( [1, 2, 3, 4, 5, 6] )

# Reshape into a 2x3 array

a = arr.reshape(2, 3)
print(a)

# Flatten the 2D array

print(a.flatten() )

# create another 2x3 array

b = np.array( [ [7, 8, 9],
                [10, 11, 12] ] )

#stack array vertically

print(np.vstack((a, b)))
```

Output:

```
[ [1  2  3]
  [4  5  6] ]

[1  2  3  4  5  6]

[ [  1   2   3]
  [  4   5   6]
  [  7   8   9]
  [10  1  12] ]
```